Assignment 4

Kumar Kanishk Singh

2024-03-28

Question 1

We fit the given model using JAGS.

```
n<-6
Y<-c(64, 13, 33, 18, 30, 20)
t <- 1:n

alpha<-dnorm(0,10000)
beta<-dnorm(0,10000)
lambda<-exp(alpha+beta*t)
data<-list(Y=Y,n=n)

library(rjags)
```

```
## Warning: package 'rjags' was built under R version 4.2.3
```

```
## Loading required package: coda
```

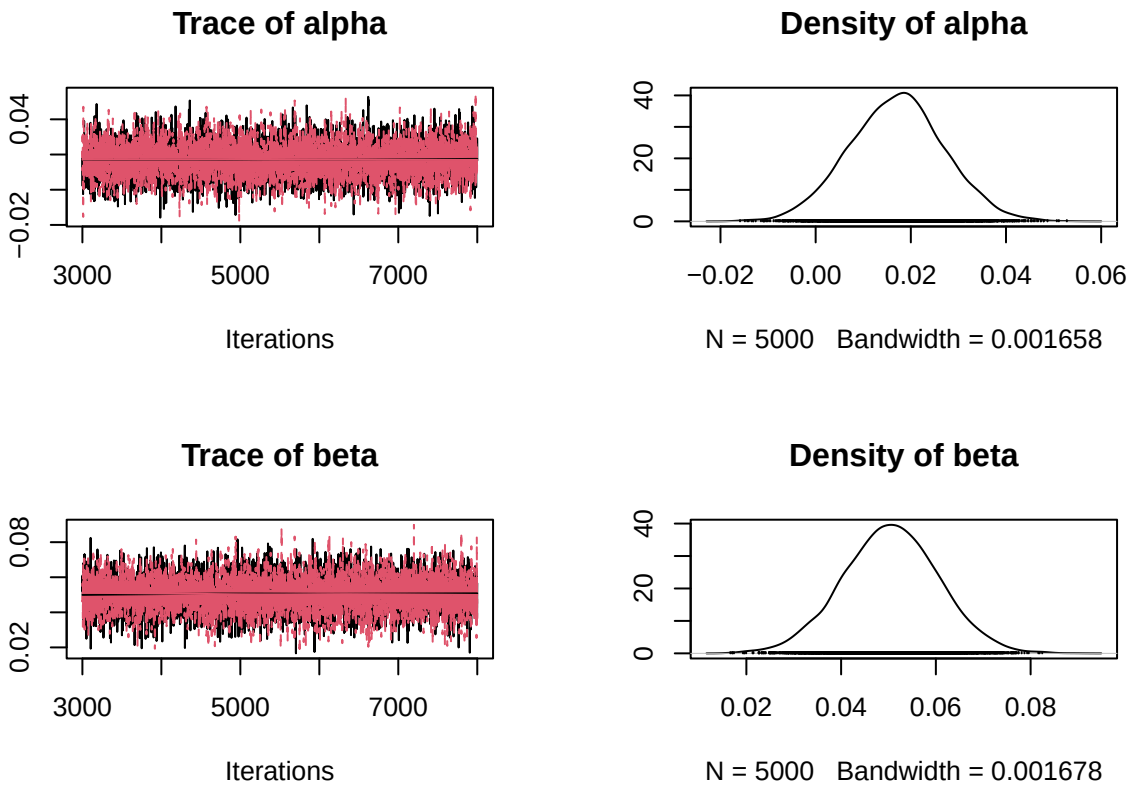
```
## Warning: package 'coda' was built under R version 4.2.3
```

```
## Linked to JAGS 4.3.1
```

```
## Loaded modules: basemod,bugs
```

```
model_string<-textConnection("model{

 #Likelihood
 for(t in 1:n){
 Y[t]~dpois(lambda[t])
 }

 #Priors
 alpha~dnorm(0,10000)
 beta~dnorm(0,10000)
 for(t in 1:n){
 lambda[t]=exp(alpha+beta*t)
 }
}")

#inits <- list(alpha=alpha, beta=beta)
model <- jags.model(model_string, data = data, quiet = TRUE, n.chains = 2)

update(model, 2000, progress.bar = "none")

params <-c("alpha","beta")
```

```
samples <- coda.samples(model,
                        variable.names = params,
                        n.iter = 5000, progress.bar = "none")

plot(samples)
```

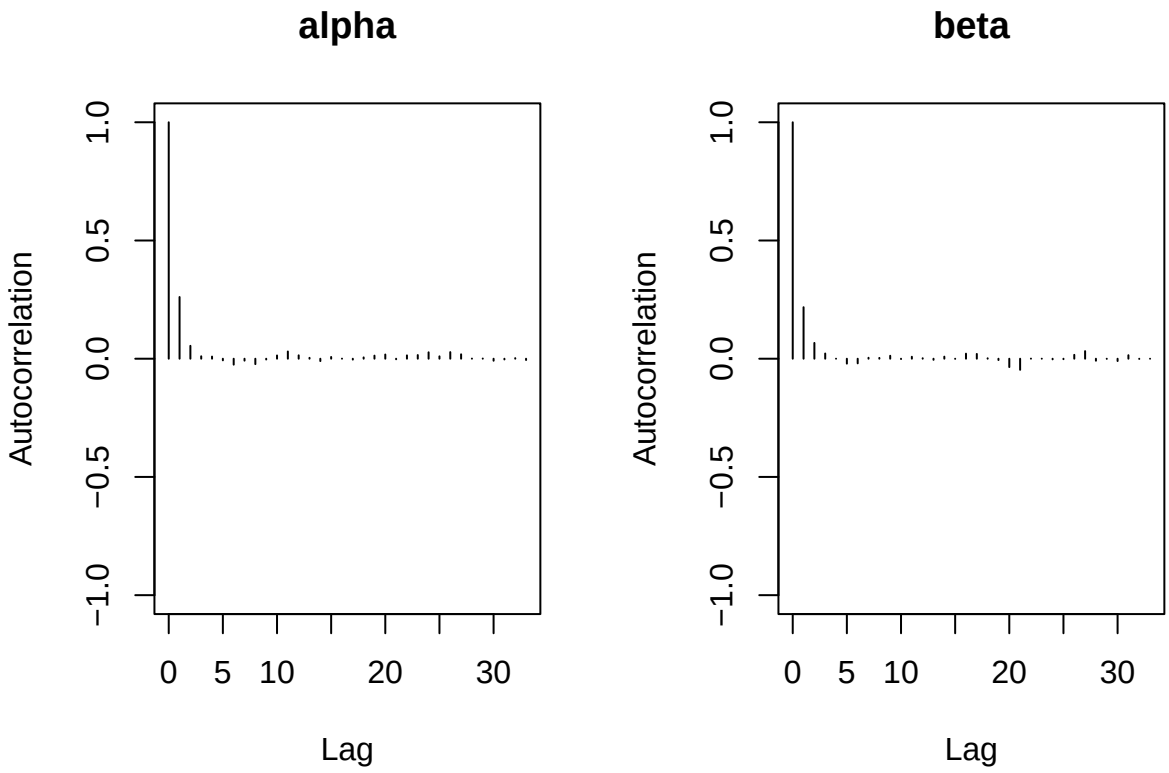


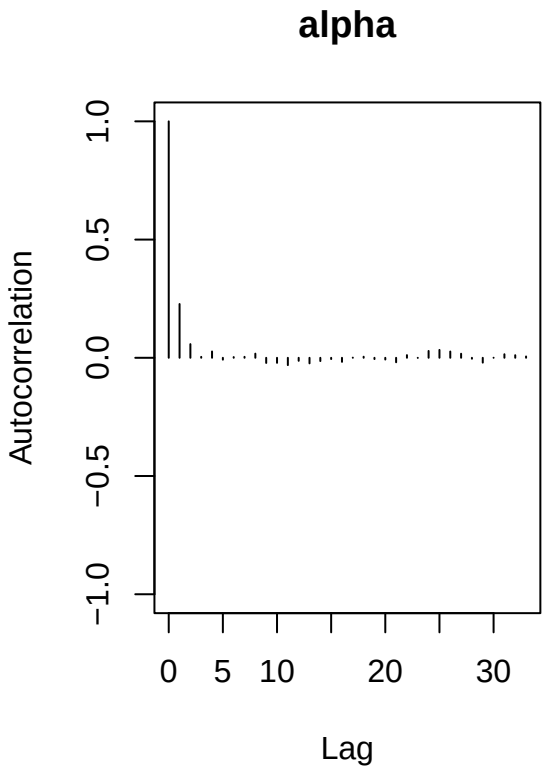

```
summary(samples)
```

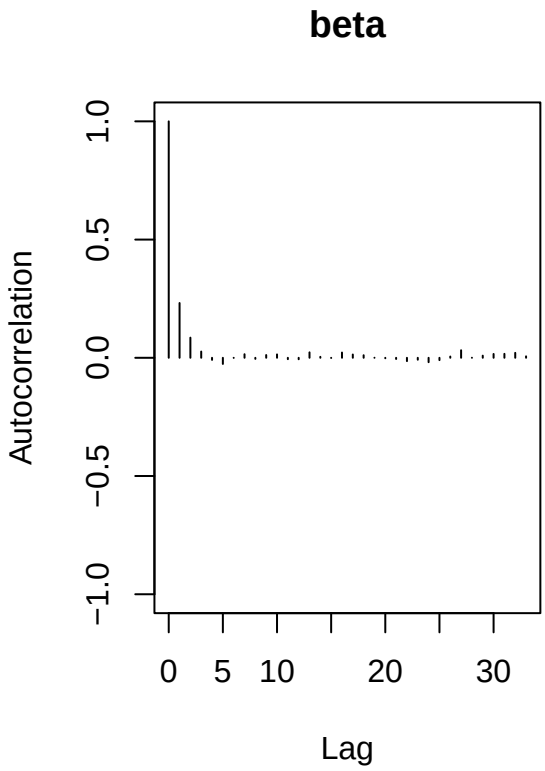
```
##
## Iterations = 3001:8000
## Thinning interval = 1
## Number of chains = 2
## Sample size per chain = 5000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##          Mean       SD  Naive SE Time-series SE
## alpha 0.01707 0.009971 9.971e-05      0.0001280
## beta  0.05048 0.010083 1.008e-04      0.0001303
##
## 2. Quantiles for each variable:
##
##            2.5%     25%     50%     75%   97.5%
## alpha -0.002502 0.01032 0.01721 0.02355 0.03661
## beta   0.030466 0.04382 0.05053 0.05720 0.07033
```

The trace plot indictes convergence. Now lets look at autocorrelation plots. The autocorrelation is low after lag 2. This also indicates convergence.

```
library(coda)

# Low autocorrelation indicates convergence
autocorr.plot(samples)
```








```
autocorr(samples, lag = 1)
```

```
## [[1]]
## , , alpha
##
##           alpha       beta
## Lag 1 0.2609655 0.01179963
##
## , , beta
##
##              alpha      beta
## Lag 1 -0.001430104 0.2178647
##
##
## [[2]]
## , , alpha
##
##           alpha         beta
## Lag 1 0.2275286 -0.002506809
##
## , , beta
##
##             alpha      beta
## Lag 1 -0.0186566 0.2317728
```

High ESS indicates convergence

```
effectiveSize(samples)
```

```
##    alpha     beta
## 6075.665 5997.645
```

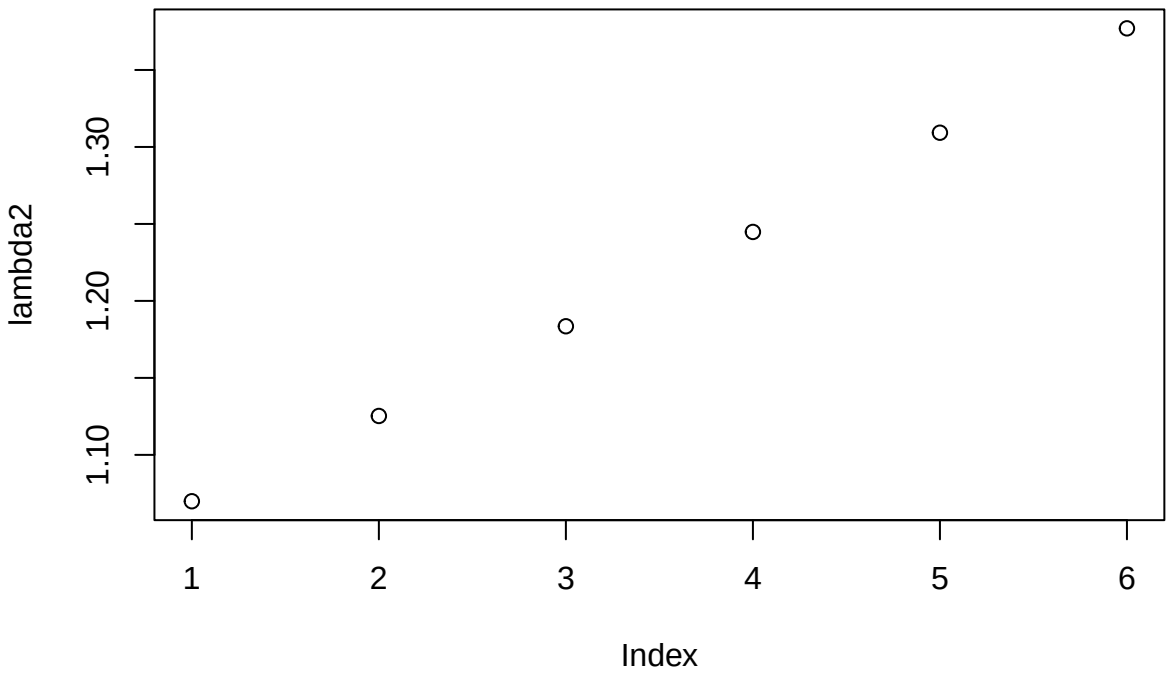
R less than 1.1 indicates convergence

```
gelman.diag(samples)
```

```
## Potential scale reduction factors:
##
##       Point est. Upper C.I.
## alpha          1          1
## beta           1          1
##
## Multivariate psrf
##
## 1
```

/z/ less than 2 indicates convergence {r] geweke.diag(samples[[1]]) We can also see that the rate of discovery increases with time.

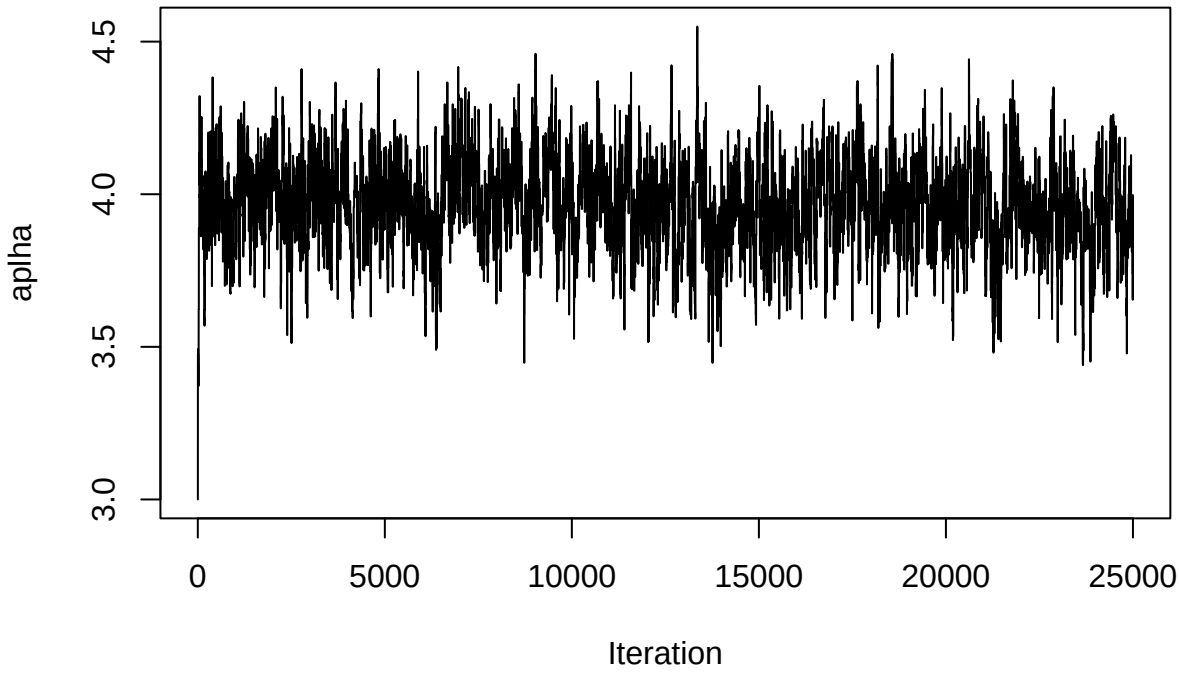
```
t<-1:n

a<-summary(samples)$statistics[1,1]
b<-summary(samples)$statistics[2,1]

lambda2<-exp(a+b*t)

plot(lambda2)
```



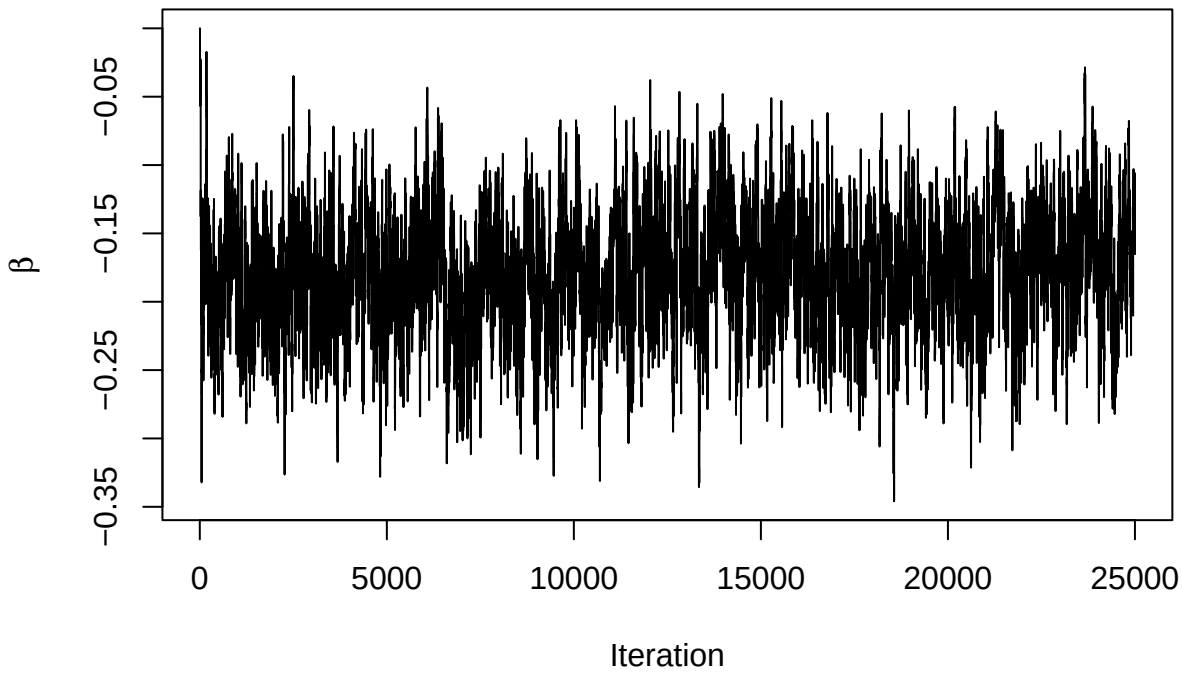
Lets set up a metropolis sampler for this problem.

```
n<-6
Y<-c(64, 13, 33, 18, 30, 20)
t <- 1:n

beta<-c(3,0)

iter<-25000
par<-matrix(0,iter,2)

cand<-c(0.4,0.06)

posterior<-function(Y,t,beta,sd=10){
  l<-exp(beta[1]+t*beta[2])
  lambda<-prod(dpois(Y,l))
  p<-prod(dnorm(beta,0,sd))
  return(lambda*p)}
#can<-rnorm(1,beta[1],cand[1])
#posterior(Y,t,can)

for(i in 1:iter)
  {
  for(j in 1:2)
  {
    can<-beta
    can[j]<-rnorm(1,beta[j],cand[j])
```

```
    R<-posterior(Y,t,can)/posterior(Y,t,beta)

    if(runif(1)<R)
    {
      beta<-can
    }
  }

  par[i,]<-beta
}

plot(par[,1],type="l",ylab=expression(aplha),xlab="Iteration")
```




```
plot(par[,2],type="l",ylab=expression(beta),xlab="Iteration")
```



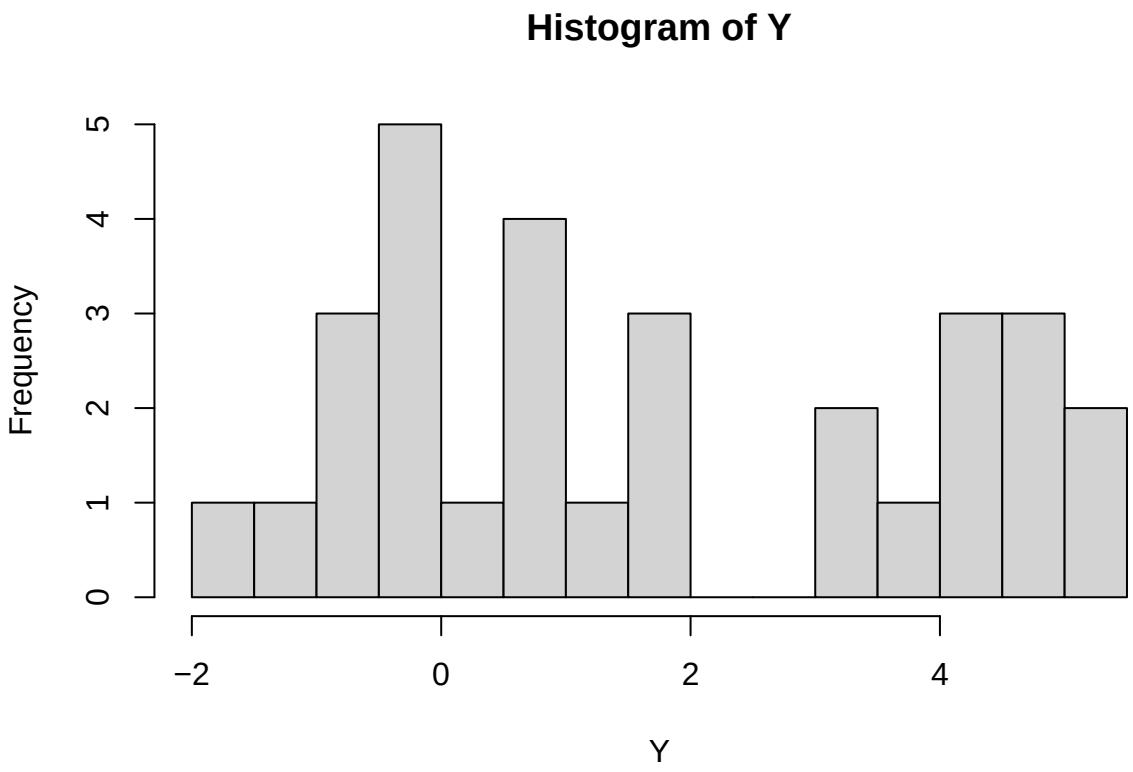
The trace plots indicate that the model has converged. Following are the acceptance ratios:

```
acc_rate <- colMeans(par[-1,]!=par[-iter,])
acc_rate
```

```
## [1] 0.2288892 0.4034561
```

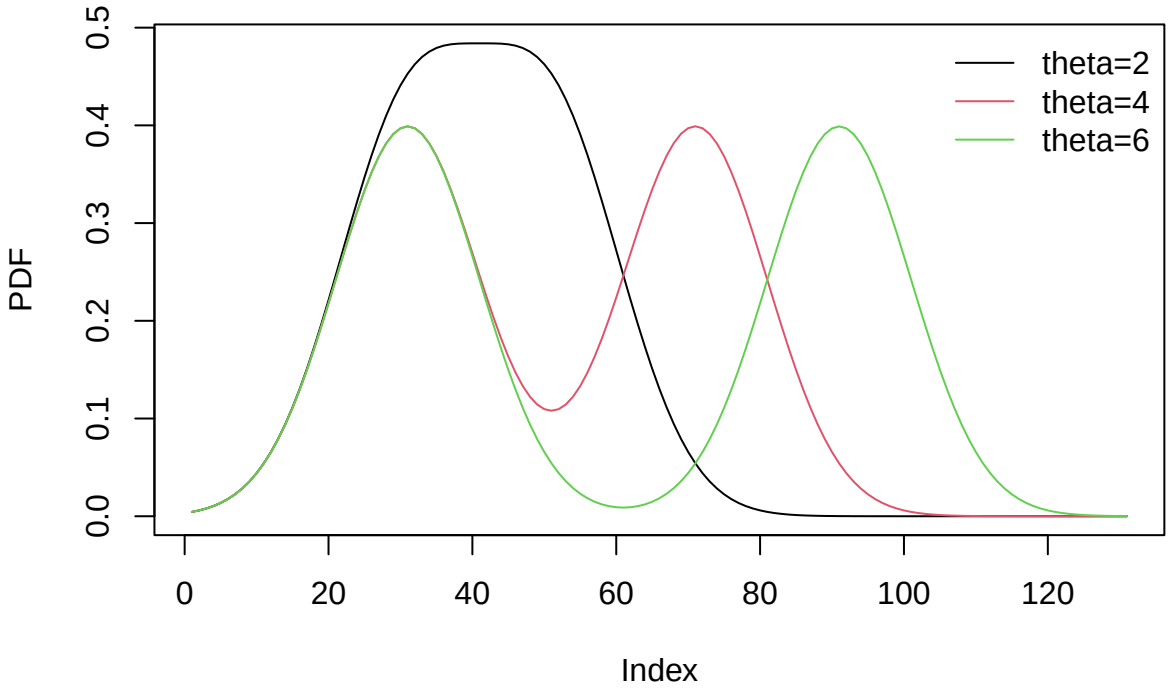
Question 2

```
set.seed(27695)
theta_true <- 4
n          <- 30
B          <- rbinom(n,1,0.5)
Y          <- rnorm(n,B*theta_true,1)
hist(Y,breaks=25)
```



b. Following are the plots for f(Y|theta) for theta={2,4,6}:

```
dens<-function(y)
{
  return(exp(-(y^2)/2)/sqrt(2*pi))
}
func<-function(y,theta)
{
  return(dens(y)+dens(y-theta))
}

y<-seq(-3,10,0.1)

plot(func(y,2),type="l",ylab="PDF")
lines(func(y,4),col=2)
lines(func(y,6),col=3)
legend("topright",c("theta=2","theta=4","theta=6"),lty=1,col=1:3,bty="n")
```



c. Following are the MAP estimate of theta and its standard deviation:

```
#install.packages("stats4")
library(stats4)
nlp <- function(theta,Y){
  like<- 0.5*dnorm(Y,0,1) + 0.5*dnorm(Y,theta,1)
  prior<- dnorm(theta,0,10)
  neg_log_post <--sum(log(like))- log(prior)
  return(neg_log_post)}
#map_est <- mle(nlp,start=list(theta=1), fixed=list(Y=Y))
#sd<- sqrt(vcov(map_est))
out <- suppressWarnings(optim(par = 1, nlp, Y = Y, hessian = T))

map_est <- out$par
sd <- sqrt(1 /c(out$hessian))
map_est; sd
```

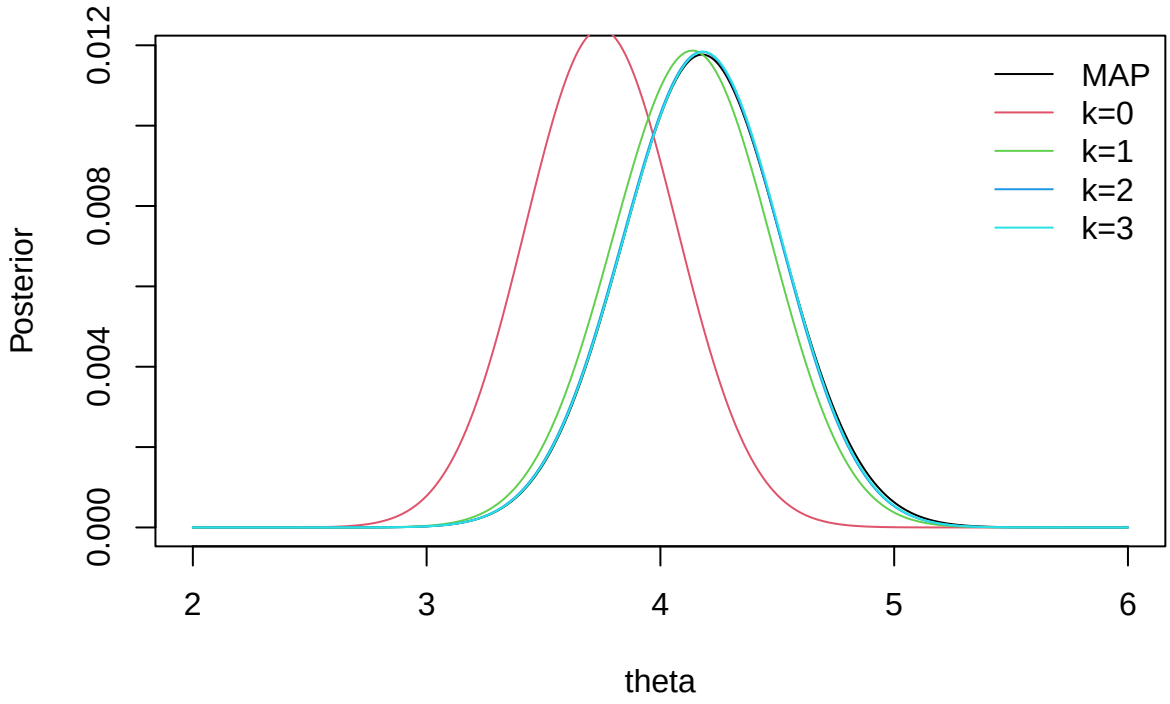
```
## [1] 4.178906
```

```
## [1] 0.3389554
```

d. Following are the posterior densities of theta:

```
posterior <- function(theta,Y,k){
  post <- dnorm(theta,0,sqrt(10^k))
  for(i in 1:length(Y)){
    post<-post*(0.5*dnorm(Y[i],0,1)+
                  0.5*dnorm(Y[i],theta,1))
  }
```

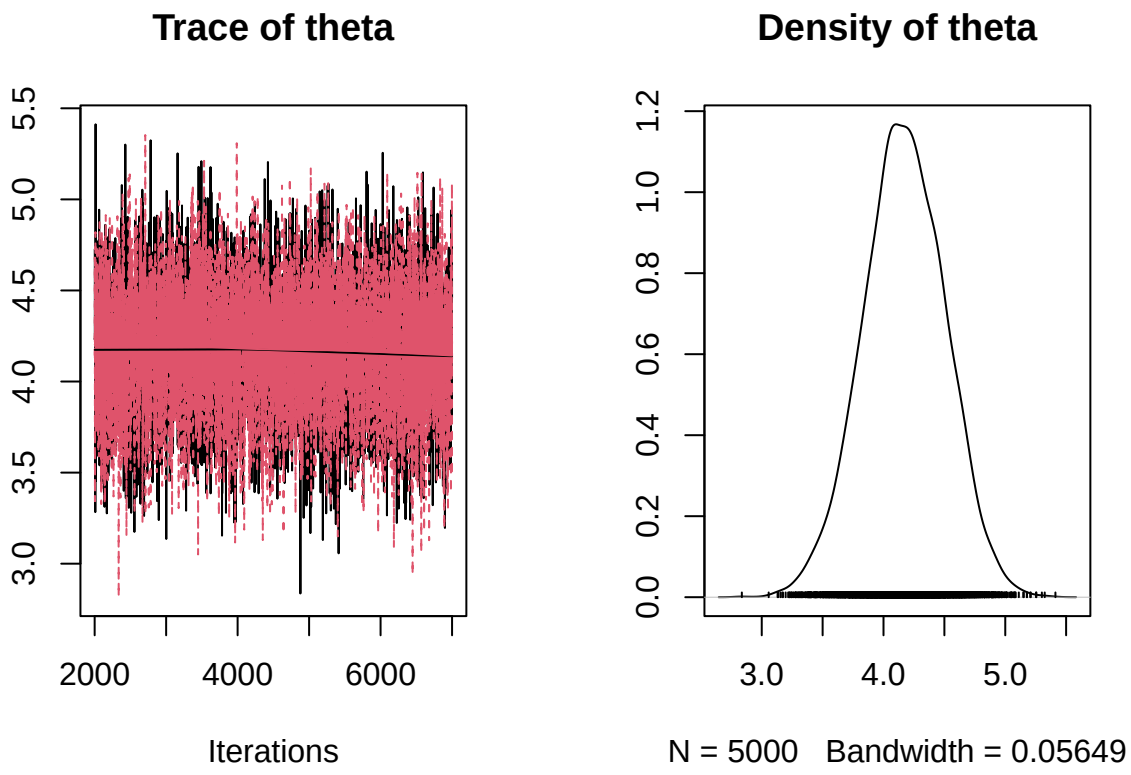
```
  return(post/sum(post))}

theta <- seq(2,6,0.01)
map <- dnorm(theta,map_est,sd)

plot(theta,map/sum(map),col=1,type="l",ylab="Posterior")
lines(theta,posterior(theta,Y,0),col=2)
lines(theta,posterior(theta,Y,1),col=3)
lines(theta,posterior(theta,Y,2),col=4)
lines(theta,posterior(theta,Y,3),col=5)
legend("topright",c("MAP","k=0","k=1","k=2","k=3"),
       col=1:5,lty=1,bty="n")
```



e.Now we use JAGS to fit this model:

```
data<-list(Y=Y,n=n)

library(rjags)


model_string<-textConnection("model{

 #Likelihood
 for(i in 1:n){
 Y[i]~dnorm(B[i]*theta,1)
 }
```

```
 #Priors
 for(i in 1:n){
 B[i]~dbern(0.5)
 }
 theta~dnorm(0,1e-2)
}")

#inits <- list(alpha=alpha, beta=beta)
model <- jags.model(model_string, data = data, quiet = TRUE, n.chains = 2)

update(model, 2000, progress.bar = "none")

params <-c("theta")
samples <- coda.samples(model,
                        variable.names = params,
                        n.iter = 5000, progress.bar = "none")

plot(samples)
```




```
summary(samples)
```

```
##
## Iterations = 2001:7000
## Thinning interval = 1
## Number of chains = 2
## Sample size per chain = 5000
```

```
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##           Mean             SD       Naive SE Time-series SE
##       4.170160       0.336226       0.003362       0.004322
##
## 2. Quantiles for each variable:
##
##  2.5%   25%   50%   75% 97.5%
## 3.499 3.946 4.170 4.403 4.823
```

We can see that in both part d and e the mean is around 4 and the height of the curve is slightly less than the 0.012.

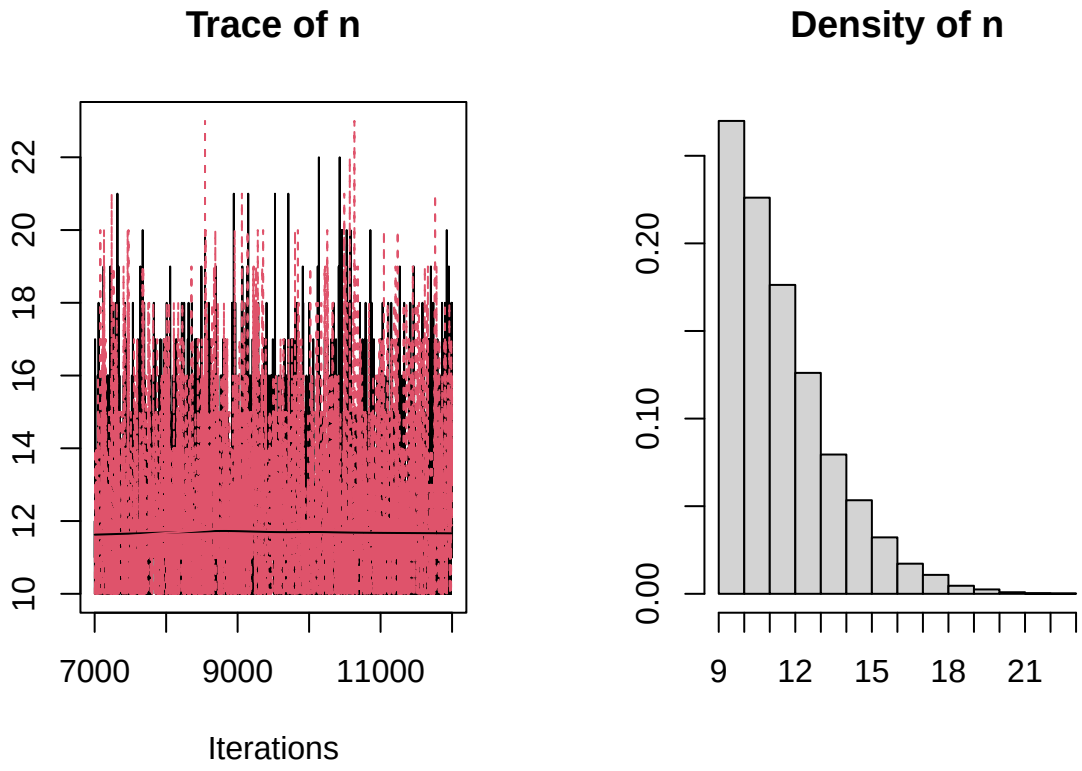
Question 3

a.Convergence could be slow because there are more parameters than observations and so it is likely that not all parameters are identifiable.

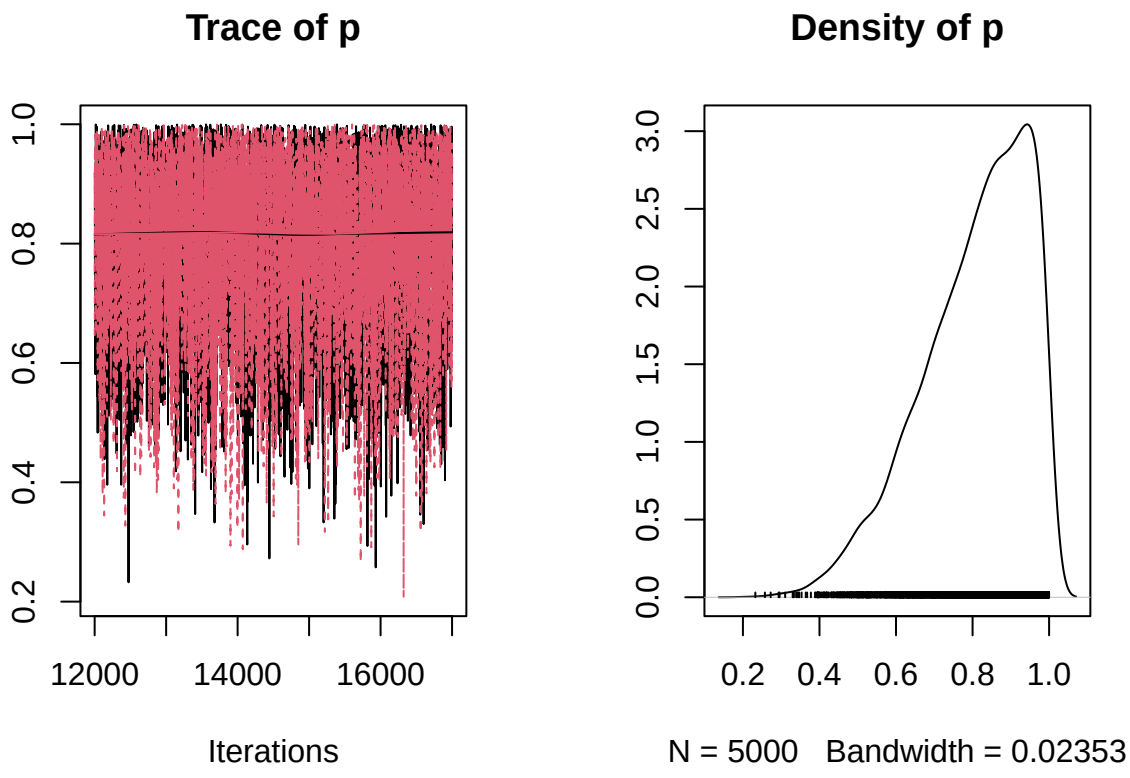
b. We set up a model using JAGS with lambda = 10 and a = b = 1.

```
Y<-10

data<-list(Y=Y)


library(rjags)


model_string<-textConnection("model{

 #Likelihood
 Y~dbin(p,n)

 #Priors
 lambda<-10
  a<-1
  b<-1
 n~dpois(lambda)
 p~dbeta(a,b)
 np<-n*p
}")

#inits <- list(n=20, p=0.5)
model <- jags.model(model_string, data = data, quiet = TRUE, n.chains = 2)

update(model, 2000, progress.bar = "none")

params <-c("n","p","np")

samples <- coda.samples(model,
                        variable.names = params,
                        n.iter = 5000, progress.bar = "none")
samples_n <- coda.samples(model,
                       variable.names = "n",
```

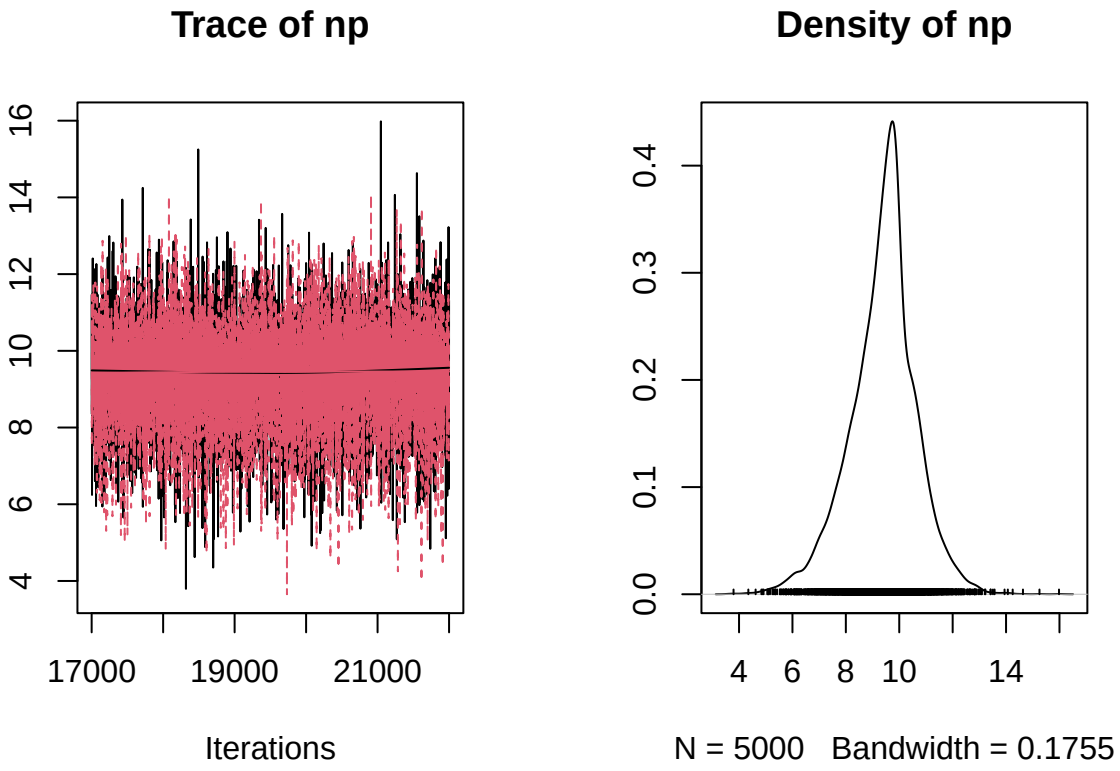
```
                          n.iter = 5000, progress.bar = "none")
samples_p <- coda.samples(model,
                          variable.names = "p",
                          n.iter = 5000, progress.bar = "none")
samples_np <- coda.samples(model,
                          variable.names = "np",
                          n.iter = 5000, progress.bar = "none")

plot(samples_n)
```




```
plot(samples_p)
```



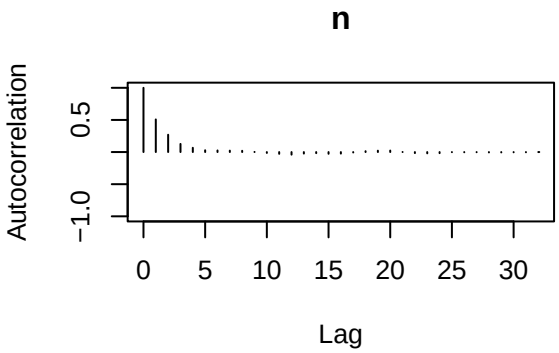

```
plot(samples_np)
```

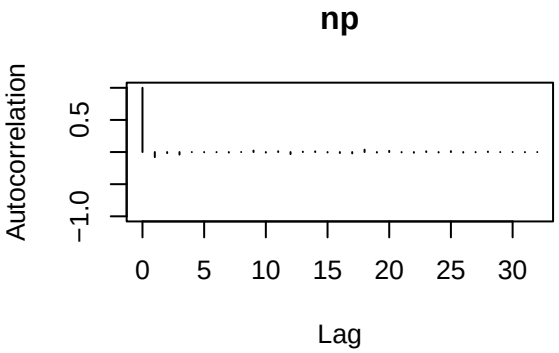


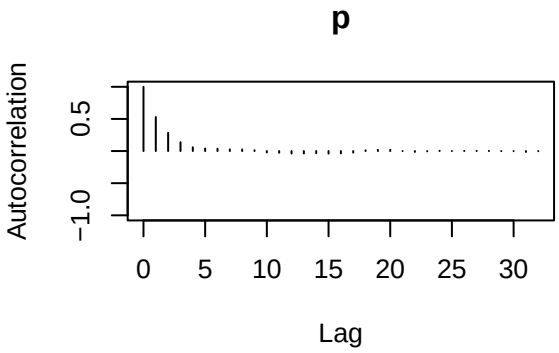

```
summary(samples)
```

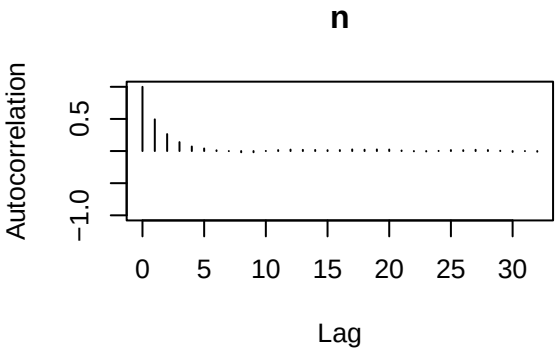
```
##
## Iterations = 2001:7000
## Thinning interval = 1
## Number of chains = 2
## Sample size per chain = 5000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##       Mean     SD Naive SE Time-series SE
## n  12.0281 2.0204 0.020204        0.03552
## np  9.3848 1.2480 0.012480        0.01084
## p   0.7972 0.1419 0.001419        0.00253
##
## 2. Quantiles for each variable:
##
##      2.5%     25%     50%     75%   97.5%
## n  10.000 10.0000 12.0000 13.0000 17.0000
## np  6.630  8.6873  9.4947 10.0954 11.7851
## p   0.478  0.7024  0.8211  0.9142  0.9918
```

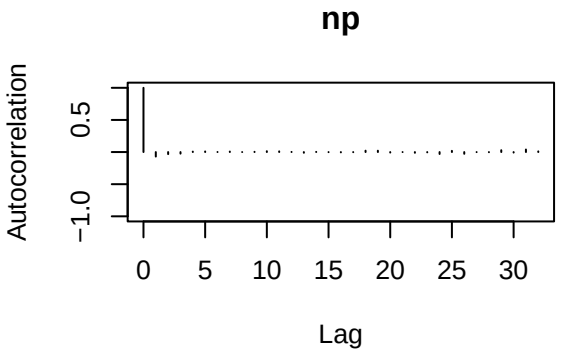
```
library(coda)

# Low autocorrelation indicates convergence
autocorr.plot(samples)
```

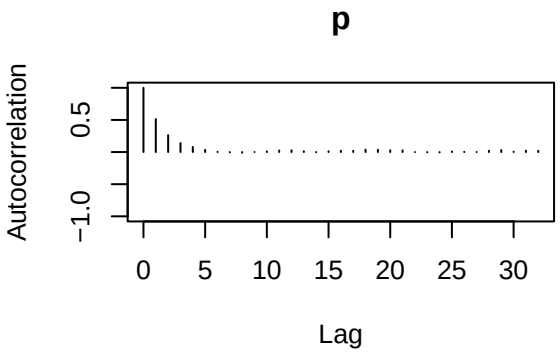

n
Autocorrelation
−1.0
0.5
0
5
10
15
20
25
30
Lag


np
Autocorrelation
−1.0
0.5
0
5
10
15
20
25
30
Lag


p
Autocorrelation
−1.0
0.5
0
5
10
15
20
25
30
Lag


n
Autocorrelation
−1.0
0.5
0
5
10
15
20
25
30
Lag






```
autocorr(samples, lag = 1)
```

```
## [[1]]
## , , n
##
##               n         np          p
## Lag 1 0.5081102 -0.4077006 -0.7195979
##
## , , np
##
##                n          np          p
## Lag 1 0.07046198 -0.07864312 -0.1138607
##
## , , p
##
##                n        np         p
## Lag 1 -0.3819104 0.2776173 0.5293689
##
##
## [[2]]
## , , n
##
##               n         np          p
## Lag 1 0.4937858 -0.4030797 -0.7030358
##
## , , np
```

```
##
##                n          np          p
## Lag 1 0.08102047 -0.07055323 -0.1139348
##
## , , p
##
##                n        np        p
## Lag 1 -0.3636065 0.2765169 0.513316
```

```
# high ESS indicates convergence
effectiveSize(samples)
```

```
##         n        np         p
##  3236.126 13257.341  3146.021
```

```
# R less than 1.1 indicates convergence
gelman.diag(samples)
```

```
## Potential scale reduction factors:
##
##    Point est. Upper C.I.
## n           1          1
## np          1          1
## p           1          1
##
## Multivariate psrf
##
## 1
```

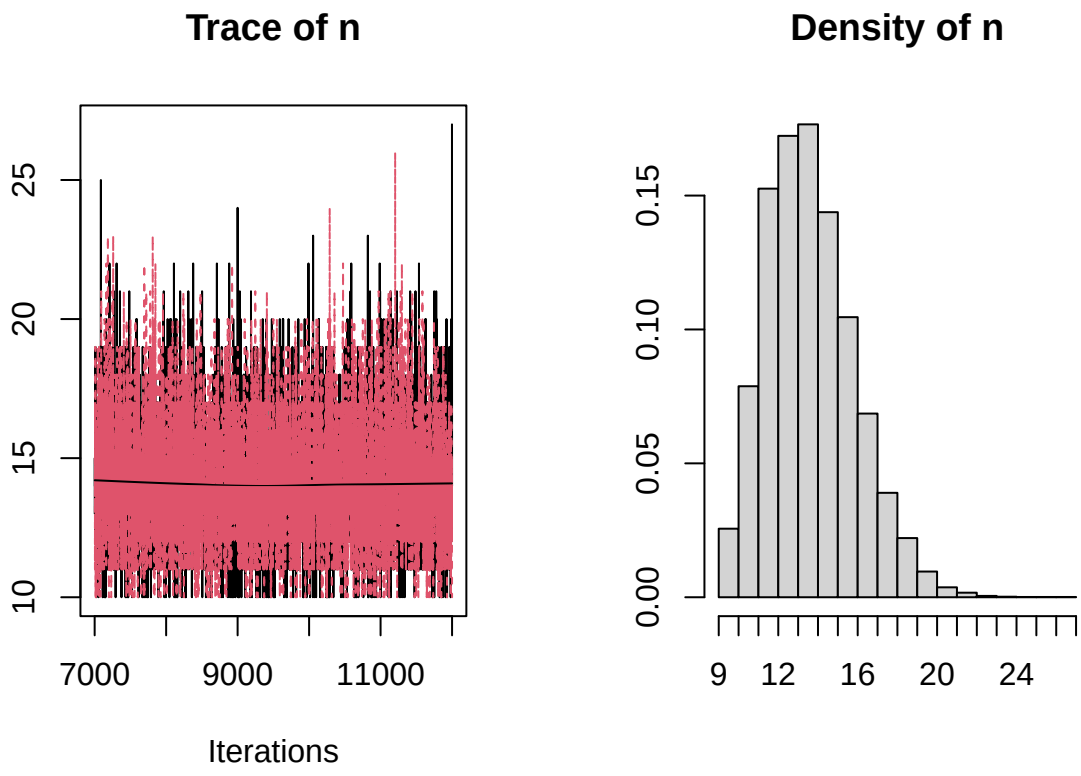
```
# /z/ less than 2 indicates convergence
geweke.diag(samples[[1]])
```

```
##
## Fraction in 1st window = 0.1
## Fraction in 2nd window = 0.5
##
##        n       np        p
## -0.44054 -0.85251  0.02406
```

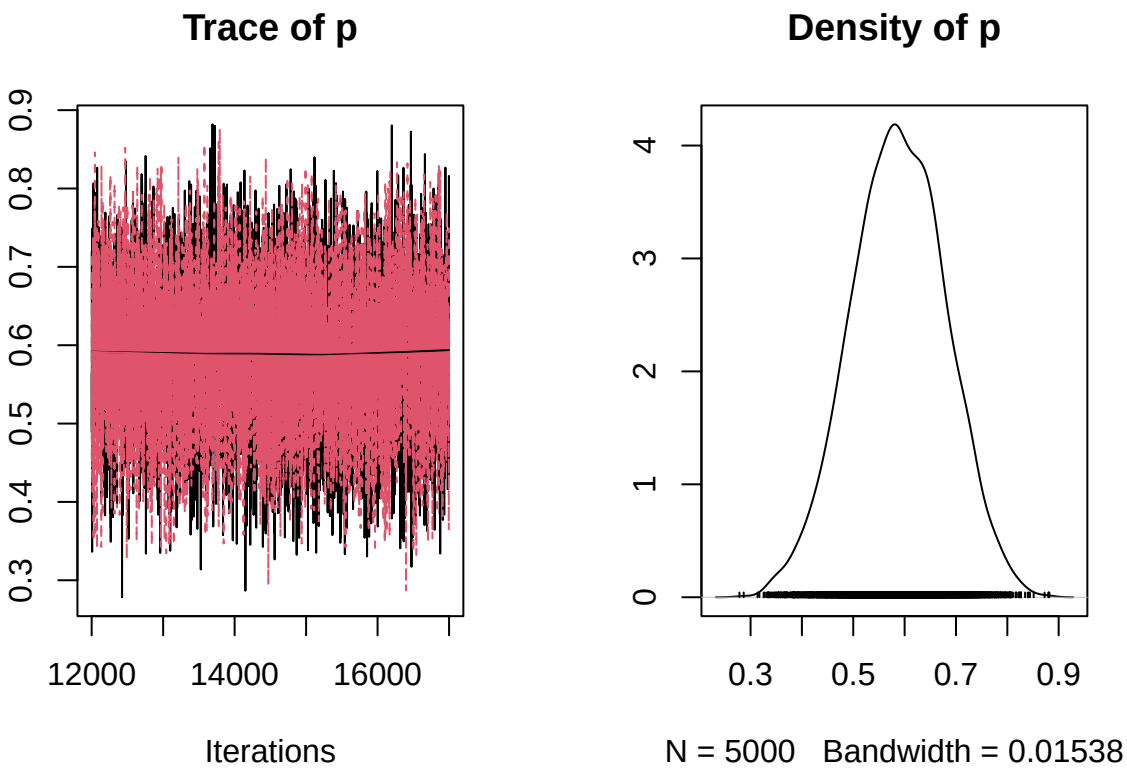
We can see the model converges by the above results. n converges at about 12, p at 0.8 and np at 10.

```
Y<-10

data<-list(Y=Y)


library(rjags)


model_string<-textConnection("model{

 #Likelihood
 Y~dbin(p,n)

 #Priors
 lambda<-10
  a<-10
```

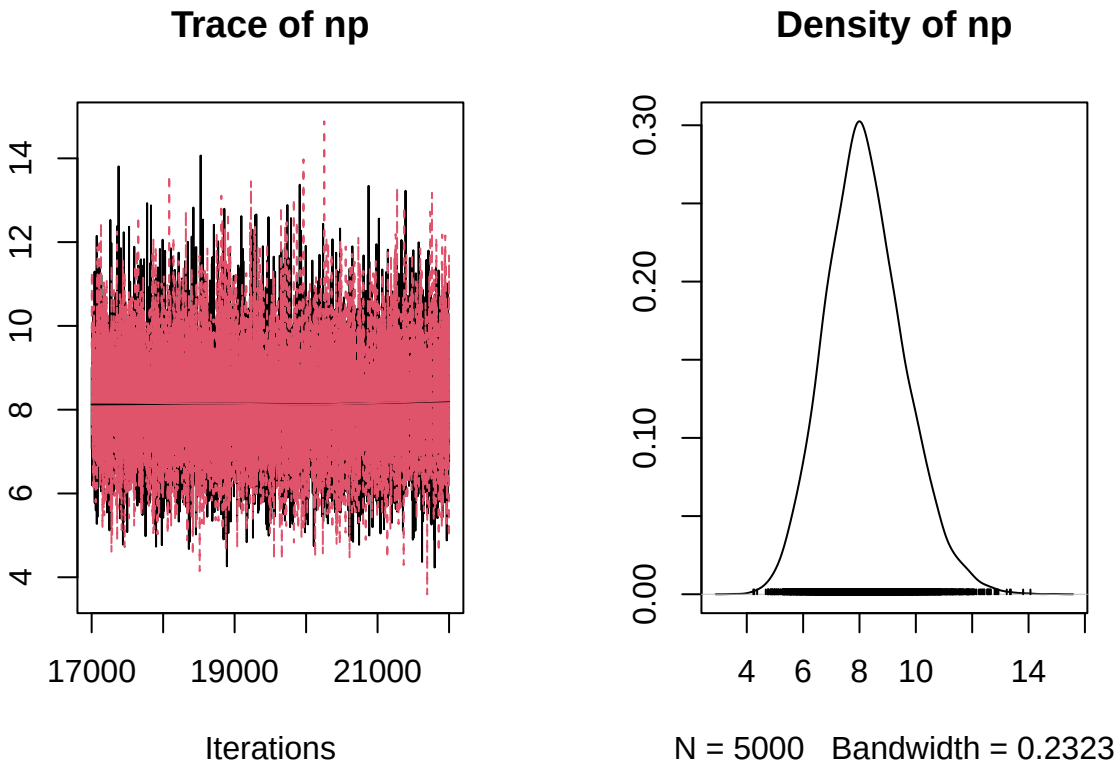
```
  b<-10
 n~dpois(lambda)
 p~dbeta(a,b)
 np<-n*p
}")

#inits <- list(n=20, p=0.5)
model <- jags.model(model_string, data = data, quiet = TRUE, n.chains = 2)

update(model, 2000, progress.bar = "none")

params <-c("n","p","np")

samples <- coda.samples(model,
                        variable.names = params,
                        n.iter = 5000, progress.bar = "none")
samples_n <- coda.samples(model,
                       variable.names = "n",
                       n.iter = 5000, progress.bar = "none")
samples_p <- coda.samples(model,
                        variable.names = "p",
                        n.iter = 5000, progress.bar = "none")
samples_np <- coda.samples(model,
                        variable.names = "np",
                        n.iter = 5000, progress.bar = "none")

plot(samples_n)
```




```
plot(samples_p)
```



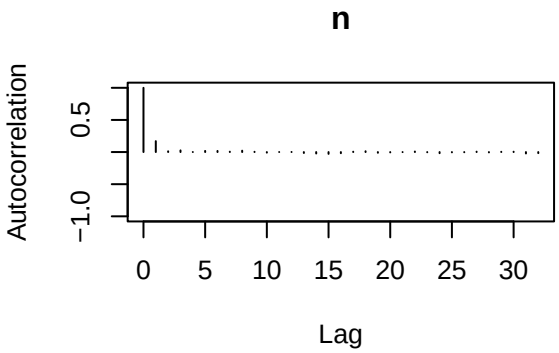

```
plot(samples_np)
```

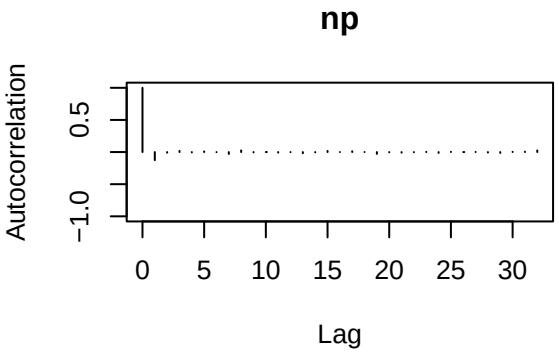


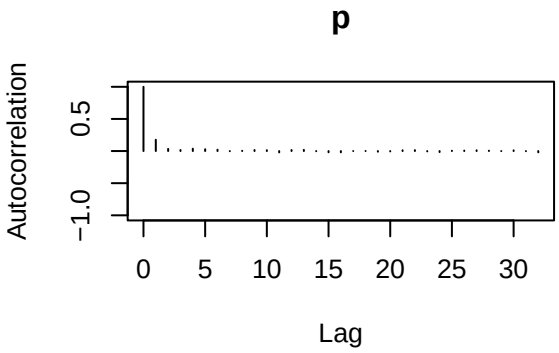

```
summary(samples)
```

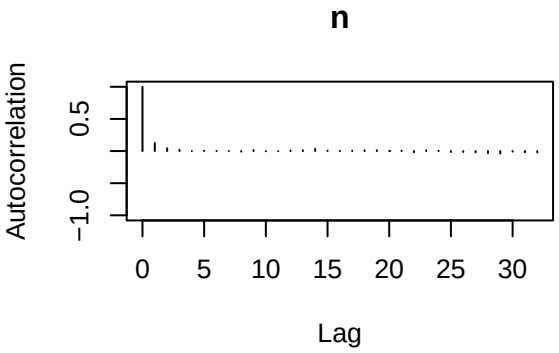
```
##
## Iterations = 2001:7000
## Thinning interval = 1
## Number of chains = 2
## Sample size per chain = 5000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##       Mean      SD  Naive SE Time-series SE
## n  14.0749 2.18453 0.0218453       0.025860
## np  8.2426 1.38798 0.0138798       0.011666
## p   0.5912 0.09018 0.0009018       0.001103
##
## 2. Quantiles for each variable:
##
##       2.5%     25%     50%     75%   97.5%
## n  11.0000 13.0000 14.0000 15.0000 19.0000
## np  5.6766  7.2882  8.1824  9.1404 11.1478
## p   0.4134  0.5303  0.5912  0.6533  0.7643
```

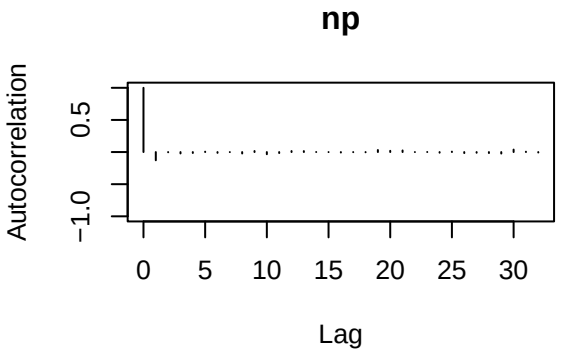
```
library(coda)

# Low autocorrelation indicates convergence
autocorr.plot(samples)
```

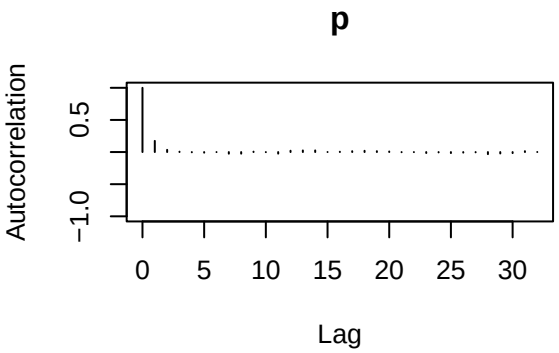

n
Autocorrelation
−1.0
0.5
0
5
10
15
20
25
30
Lag


np
Autocorrelation
−1.0
0.5
0
5
10
15
20
25
30
Lag


p
Autocorrelation
−1.0
0.5
0
5
10
15
20
25
30
Lag


n
Autocorrelation
−1.0
0.5
0
5
10
15
20
25
30
Lag






```
autocorr(samples, lag = 1)
```

```
## [[1]]
## , , n
##
##               n         np          p
## Lag 1 0.1705186 -0.2313435 -0.4239896
##
## , , np
##
##                n         np          p
## Lag 1 0.08672159 -0.1265587 -0.2265811
##
## , , p
##
##                 n         np         p
## Lag 1 -0.07578815 0.09147455 0.1760072
##
##
## [[2]]
## , , n
##
##               n         np          p
## Lag 1 0.1251608 -0.2393892 -0.3925335
##
## , , np
```

```
##
##                n         np          p
## Lag 1 0.05409044 -0.1279724 -0.1963544
##
## , , p
##
##                 n         np         p
## Lag 1 -0.06590293 0.09511345 0.1739781
```

```
# high ESS indicates convergence
effectiveSize(samples)
```

```
##         n        np         p
##  7143.889 14183.953  6706.204
```

```
# R less than 1.1 indicates convergence
gelman.diag(samples)
```

```
## Potential scale reduction factors:
##
##    Point est. Upper C.I.
## n           1          1
## np          1          1
## p           1          1
##
## Multivariate psrf
##
## 1
```

```
# /z/ less than 2 indicates convergence
geweke.diag(samples[[1]])
```

```
##
## Fraction in 1st window = 0.1
## Fraction in 2nd window = 0.5
##
##      n     np      p
## 0.4187 0.9367 0.3424
```

We can see the model converges by the above results. n converges at about 14, p at 0.6 and np at 8.

Question 4

We set up a two sample t-test to test whether the treatment is effective or not. We take Yi~Normal(mu,sigma^2) for placebo observations and Yi~Normal(mu+delta,sigma^2) for treatment observations. We test Whether delta=0 or not.

```
n<-12
n1<-6
n2<-6
Y1<-c(2.0,-3.1,-1.0,0.2,0.3,0.4)
Y2<-c(-3.5,-1.6,-4.6,-0.9,-5.1,0.1)

Y1_m<-mean(Y1)
Y2_m<-mean(Y2)

sigma1<-mean((Y1_m-Y1)^2)
```

```r
sigma2<-mean((Y2_m-Y2)^2)
sig_rt<-sqrt(sigma1/2+sigma2/2)

p_mean<-Y2_m-Y1_m
sigma_hat<-(sig_rt)*sqrt(1/n1+1/n2)


cred_set <- p_mean+sigma_hat*qt(c(0.025,0.975),df=n1+n2)


p_mean;sigma_hat;cred_set
```

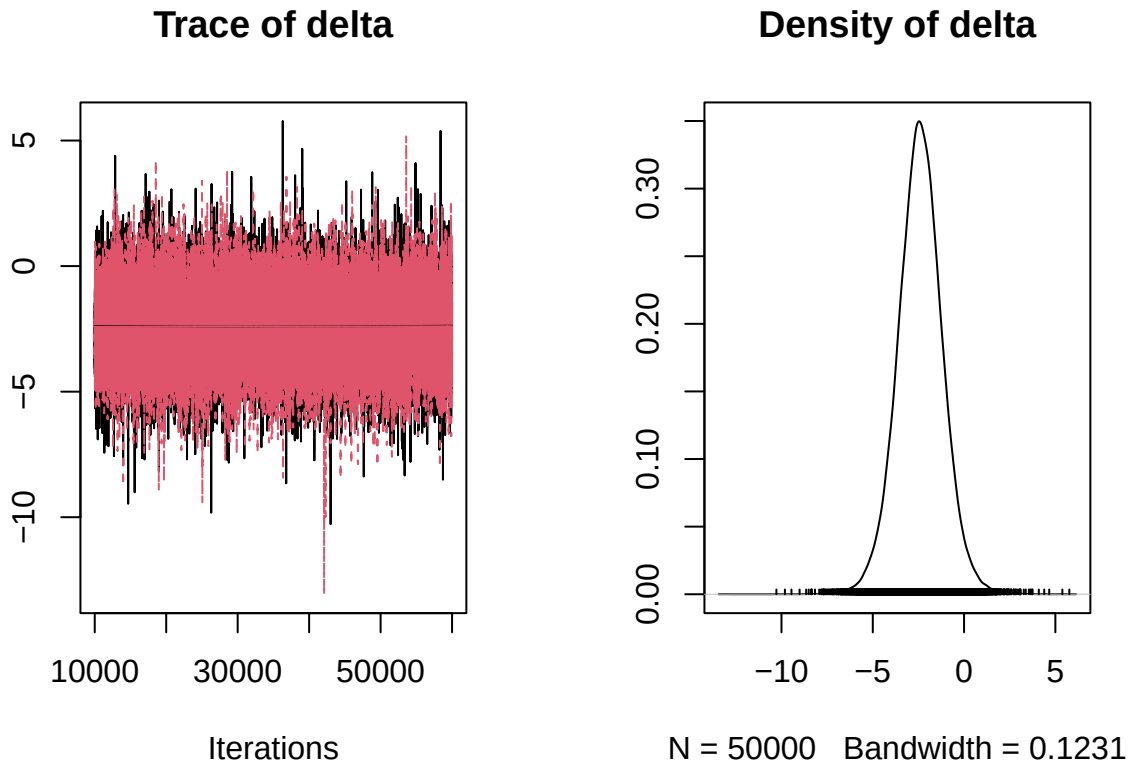
```
## [1] -2.4
```

```
## [1] 1.012423
```

```
## [1] -4.6058799 -0.1941201
```

The 95% credible set doesn't include 0. So, we can say that the treatment is effective. To test for sensitivity to the prior we also fit the model using proper priors using JAGS.

```r
library(rjags)
data <- list(n=6,Y1=Y1,Y2=Y2)

model_string <- textConnection("model{

   # Likelihood
   for(i in 1:n){
     Y1[i] ~ dnorm(mu,tau)
     Y2[i] ~ dnorm(mu+delta,tau)
   }

   # Priors
   mu    ~  dnorm(0, 0.0001)
   delta ~  dnorm(0, 0.0001)
   tau   ~  dgamma(0.1, 0.1)
   sigma <- 1/sqrt(tau)
 }")

model <- jags.model(model_string,data = data, n.chains=2,quiet=TRUE)
update(model, 10000, progress.bar="none")
params  <- c("delta")
samples <- coda.samples(model,
                        variable.names=params,
                        n.iter=50000, progress.bar="none")
plot(samples)
```




```
summary(samples)
```

```
##
## Iterations = 10001:60000
## Thinning interval = 1
## Number of chains = 2
## Sample size per chain = 50000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##           Mean             SD       Naive SE Time-series SE
##      -2.403981       1.232590       0.003898       0.006818
##
## 2. Quantiles for each variable:
##
##     2.5%      25%      50%      75%    97.5%
## -4.86353 -3.18273 -2.40437 -1.62654  0.05118
```

We can see that the 95% credible interval now includes 0. So, we can say that the conclusion is slightly sensitive to prior.